

5.2 PROGRAMAÇÃO DINÂMICA

A estratégia da programação dinâmica costuma ser aplicada a certos **problemas de otimização**, resultando, em geral algoritmos mais eficientes que os triviais, pois a cada passo são **eliminadas subsoluções** que certamente não farão parte da **solução ótima do problema**.

Esse método é especialmente útil quando não é fácil chegar a uma **seqüência ótima de decisões** sem testar todas as seqüências possíveis para então escolher a melhor.

Reduz drasticamente o número total de **seqüências viáveis** através de um mecanismo que evita aquelas que sabidamente não podem resultar ótimas.

Muitas vezes, quando o algoritmo direto tem **complexidade exponencial**, o algoritmo desenvolvido por programação dinâmica é polinomial, reduzindo drasticamente o **tempo de execução**.

Outras vezes, a complexidade continua exponencial, mas de ordem mais baixa.

De qualquer forma, a programação dinâmica é uma estratégia para projeto de **algoritmos eficientes** bastante usada (cf. exercícios adicionais propostos no fim do capítulo).

A programação dinâmica pode ser aplicada a problemas de **otimização combinatória**, como por exemplo:

- multiplicação de várias matrizes;
- caminhos de custo mínimo em grafos orientados;
- projeto de sistemas confiáveis;
- problema do caixeiro viajante (cf. exercício adicional);
- problema da partição (cf. exercício adicional);
- problema da mochila (cf. exercício adicional).

Multiplicação de matrizes.

$$M := M_1 \times M_2 \times \dots \times M_n,$$

onde cada matriz M_i tem b_{i-1} linhas e b_i colunas, para $1 \leq i \leq n$.

Para multiplicar uma matriz A $p \times q$ por outra, B $q \times r$,

$$[A \times B]_{ij} := \sum_{k=1}^q A_{ik} \cdot B_{kj} \quad (\text{para } i = 1, \dots, p \text{ e } j = 1, \dots, r)$$

requer $p \times q \times r$ multiplicações de elementos.

Considere o seguinte exemplo:

$$M := \begin{matrix} M_1 & \times & M_2 & \times & M_3 & \times & M_4 & \times & M_5 \\ 100 \times 3 & & 3 \times 10 & & 10 \times 50 & & 50 \times 30 & & 30 \times 5. \end{matrix}$$

Calcular o produto, $\{[(M_1 \times M_2) \times M_3] \times M_4\} \times M_5$, requer:

$$(100 \times 10 \times 3) + (100 \times 10 \times 50) + (100 \times 50 \times 30) + (100 \times 30 \times 5) = 218.000 \text{ operações.}$$

E calcular o produto $M_1 \times \{M_2 \times [(M_3 \times M_4) \times M_5]\}$, requer:

$$(10 \times 50 \times 30) + (10 \times 30 \times 5) + (3 \times 10 \times 5) + (100 \times 3 \times 5) = 18.150 \text{ operações}$$

A **ordem** em que são realizadas as multiplicações pode influir substancialmente no número **total de operações** requeridas.

O problema consiste, então, em determinar uma **seqüência ótima** de multiplicações, i. e. com custo mínimo.

Um algoritmo que enumere todas as seqüências possíveis de multiplicações, calcule os respectivos números totais de operações requeridas, para, só então, escolher uma seqüência ótima terá complexidade exponencial em n (número de matrizes).

Utilizando a **programação dinâmica**, poderemos diminuir a ordem de complexidade para uma ordem polinomial.

Estamos interessados no **custo mínimo** do produto das n matrizes, i. e., no número mínimo de multiplicações necessárias para calcular:

$$M := M_1 \times M_2 \times \dots \times M_n$$

Para $1 \leq i < j \leq n$, consideremos o subproblema de calcular o produto:

$$M_{ij} = \begin{matrix} M_i & x & M_{i+1} & x L & x & M_j \\ b_{i-1} x b_i & & b_i x b_{i+1} & & & b_{j-1} x b_j \end{matrix};$$

com custo (número de multiplicações) mínimo, dado por m_{ij} .

O **problema original**, correspondente à matriz M_{1n} com custo mínimo m_{1n} .

Para $i \leq k < j$, vamos considerar os produtos:

$$M_{ik} := M_i x M_{i+1} x \dots x M_k \text{ e}$$

$$M_{(k+1)j} := M_{k+1} x M_{k+2} x \dots x M_j.$$

Notemos que essas matrizes M_{ik} e $M_{(k+1)j}$ correspondem a **subproblemas** do problema correspondente à matriz M_{ij} e, portanto, subproblemas do problema original correspondente à matriz M .

Essas duas matrizes M_{ik} e $M_{(k+1)j}$, respectivamente, têm:

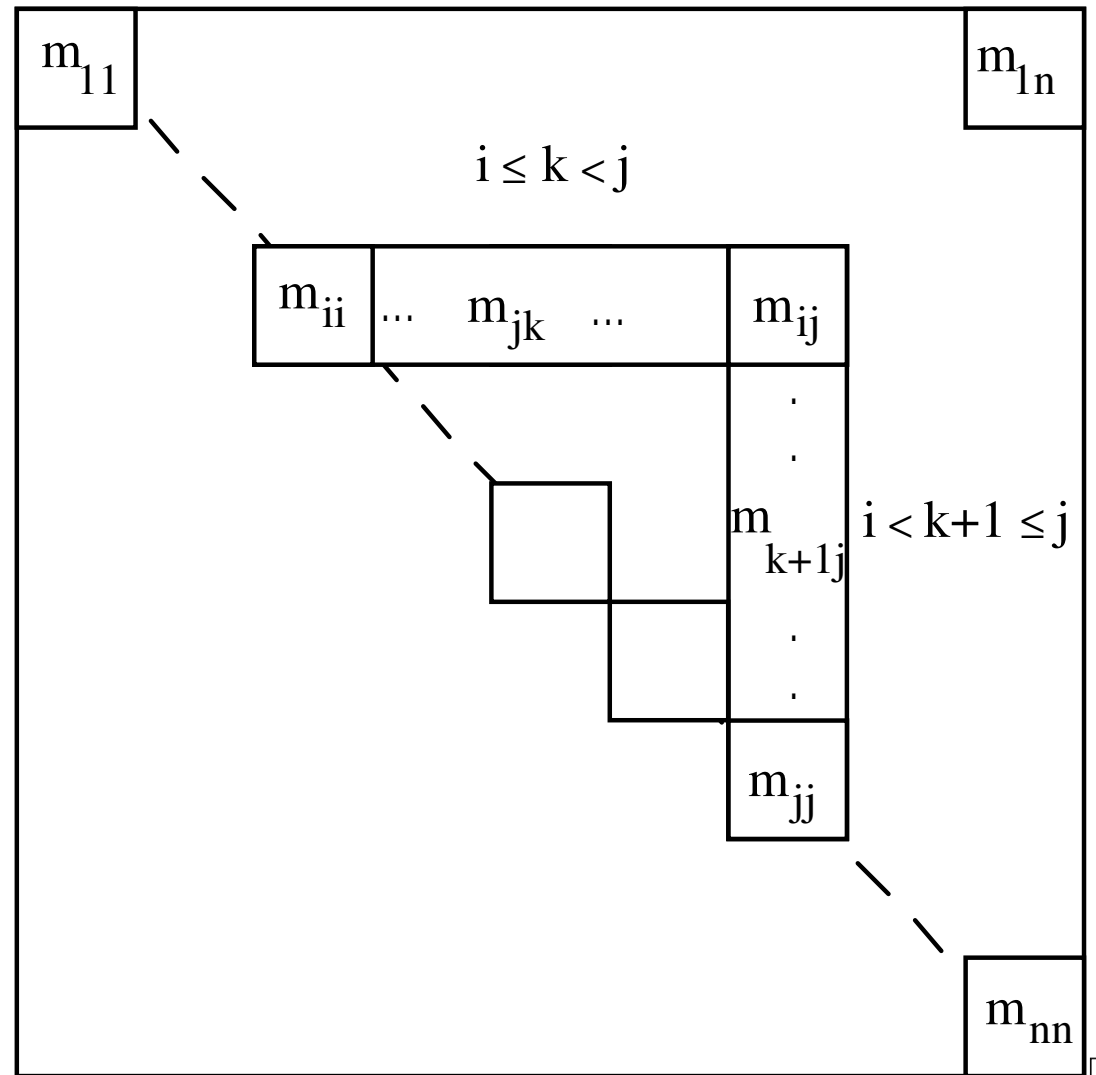
- custo mínimo m_{ik} e dimensões $b_{i-1} \times b_k$ (para M_{ik}) e
- custo mínimo $m_{(k+1)j}$ e dimensões $b_k \times b_j$ (de $M_{(k+1)j}$).

O produto M_{ij} pode ser obtido, fazendo $M_{ik} \times M_{(k+1)j}$, e o custo será dado por
 $(m_{ik} + m_{(k+1)j}) + (b_{i-1} \times b_k \times b_j)$.

Para o custo ser mínimo, tem-se que escolher um k apropriado, ou seja:

$$m_{ij} = \min_{i \leq k < j} \{ (m_{ik} + m_{(k+1)j}) + (b_{i-1} \times b_k \times b_j) \}.$$

Como $i \leq k < j$, tem-se $k-i < j-i$ e $j-(k+1) < j-1$, portanto os **custos mínimos** m_{ik} e $m_{(k+1)j}$ já terão sido obtidos antes. Assim, como mostra a figura 5.3.3, não há necessidade de recursão.



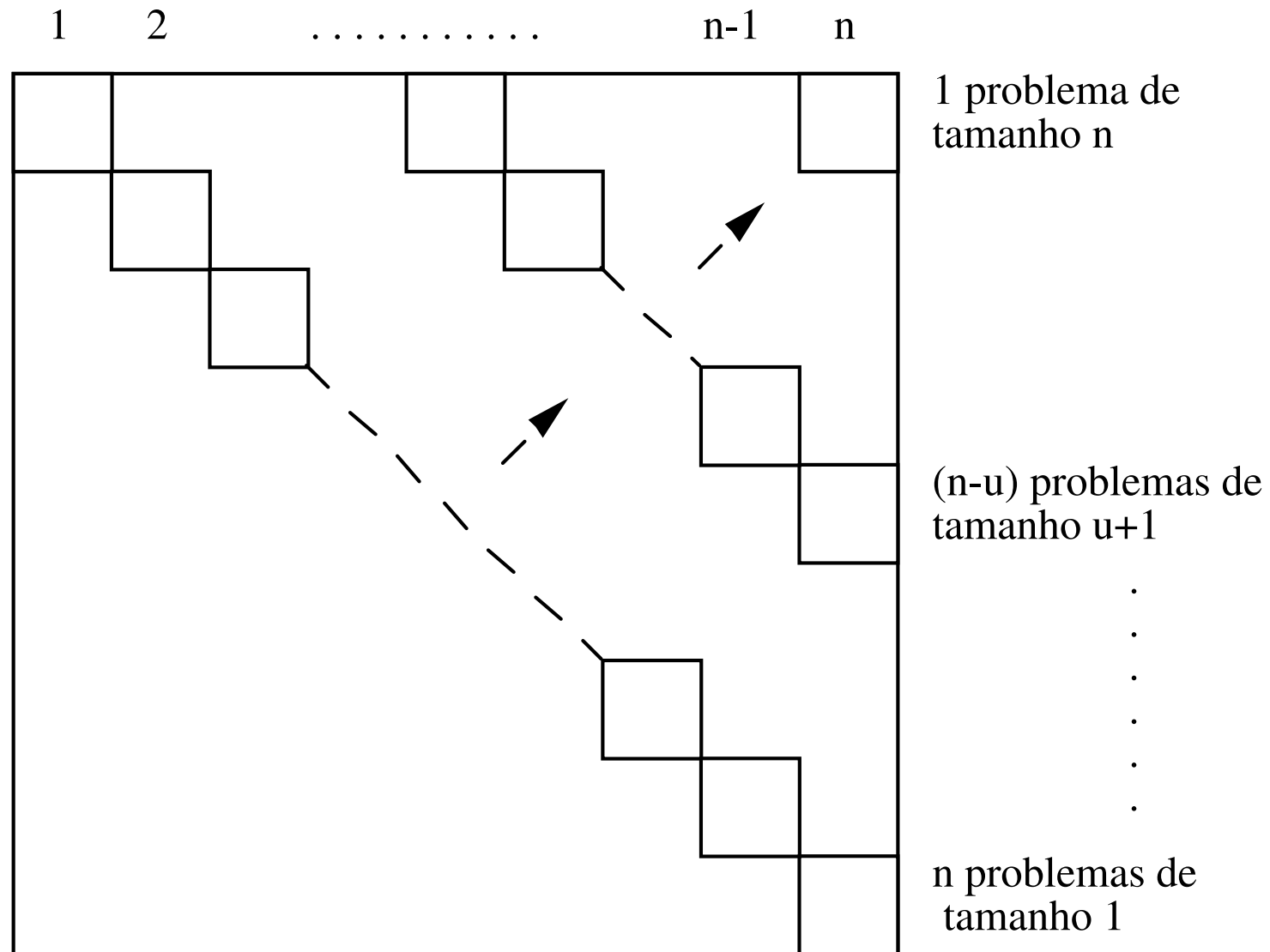
Processo de cálculo de custo mínimo de produto de matrizes

Um algoritmo para resolver esse problema de custo mínimo do produto de n matrizes por programação dinâmica utiliza como área auxiliar para armazenar os **custos intermediários** uma matriz quadrada.

O algoritmo funciona da seguinte maneira:

1. Num primeiro passo, a diagonal dessa matriz é inicializada: $m_{ii} := 0$, para $1 \leq i \leq n$.
2. Passo a passo, são obtidos valores correspondentes às diagonais superiores da matriz.
3. Finalmente, o processo termina com o valor m_{1n} no canto superior direito.

Desse modo, podemos formular um algoritmo para determinar o custo mínimo para calcular o produto de várias matrizes.



Hierarquia de instâncias na programação dinâmica

Função: Multi_Mat($b:D$) $\rightarrow$ IN {custo mínimo de produto de matrizes}

{ *Entrada:* vetor $b:D$ (para $i = 1, \dots, n$, das n matrizes $M_i : b[i-1] \times b[i]$)

Saída: $m[1, n]:IN$ (n^o mín. de mult. para o produto $M_1 \times M_2 \times \dots \times M_n$) }

sorte $D :=$ vetor $[0..n]$ de IN {entrada} ; $M :=$ matriz $[1..n, 1..n]$ de IN {auxiliar} ;

var {auxiliares} $m:M$ {matriz auxiliar: $m[i, j]$ ($1 \leq i \leq j \leq n$) (custo mínimo para o produto $M_i \times \dots \times M_j$)} ; $j:IN$ {índice de matriz} ;

1. para i de 1 até n faça $m[i, i] \leftarrow 0$; {inicializa diagonal principal}
2. para u de 1 até $n-1$ faça {deslocamento da diagonal: 7}
3. para i de 1 até $n-u$ faça {posição na diagonal: 6}
4. $j \leftarrow i+u$; { $u=j-i$ } ;
5. $m[i, j] \leftarrow \min_{i \leq k < j} \{ (m[i, k] + m[k+1, j]) + (b[i-1] \times b[k] \times b[j]) \}$;
6. fim-para {3: i de 1 até $n-u$ }
7. fim-para {2: u de 1 até $n-1$ }
8. retorne-saída($m[1, n]$) ; {dá saída extraída}
9. fim-Função {fim do algoritmo Multi_Mat: custo mínimo de produto}

A **seqüência ótima das multiplicações** pode ser obtida, guardando-se os valores de k , obtidos na linha 5 do algoritmo.

A **complexidade pessimista** do algoritmo Multi_Mat tem **ordem cúbica**, portanto a programação dinâmica tornou possível resolver com uma complexidade de ordem polinomial um problema para o qual o algoritmo imediato teria complexidade de ordem exponencial.

Lembrando que uma iteração definida tem comportamento equivalente a uma iteração definida inicializada, podemos simular a iteração definida

para u de 1 até $n - 1$ (das linhas de 2 a 7)

de Multi_Mat por:

$u \leftarrow 0$; enqto $u < n$ faça $u \leftarrow u + 1$; corpo {linhas de 4 a 5} fim-enqto.

Assim, podemos reescrever o texto de Multi_Mat como se segue:

1. para i de 1 até n faça $m[i, i] \leftarrow 0$; {inicializa diagonal principal}
- 1.a $u \leftarrow 0$; {inicializa controle u}
2. enqto $u < n$ faça {deslocamento da diagonal: 7}
- 2.a $u \leftarrow u + 1$; {incrementa controle u}
3. para i de 1 até $n - u$ faça {posição na diagonal: 6}
4. $j \leftarrow i + u$; { $u = j - i$ }
5. $m[i, j] \leftarrow \min_{i \leq k < j} \{ (m[i, k] + m[k + 1, j]) + (b[i - 1] \times b[k] \times b[j]) \}$;
6. fim-para {3: i de 1 até $n - u$ }
7. fim-enqto {2: $u < n$ }
8. retorne-saída($m[1, n]$) ; {dá saída extraída}

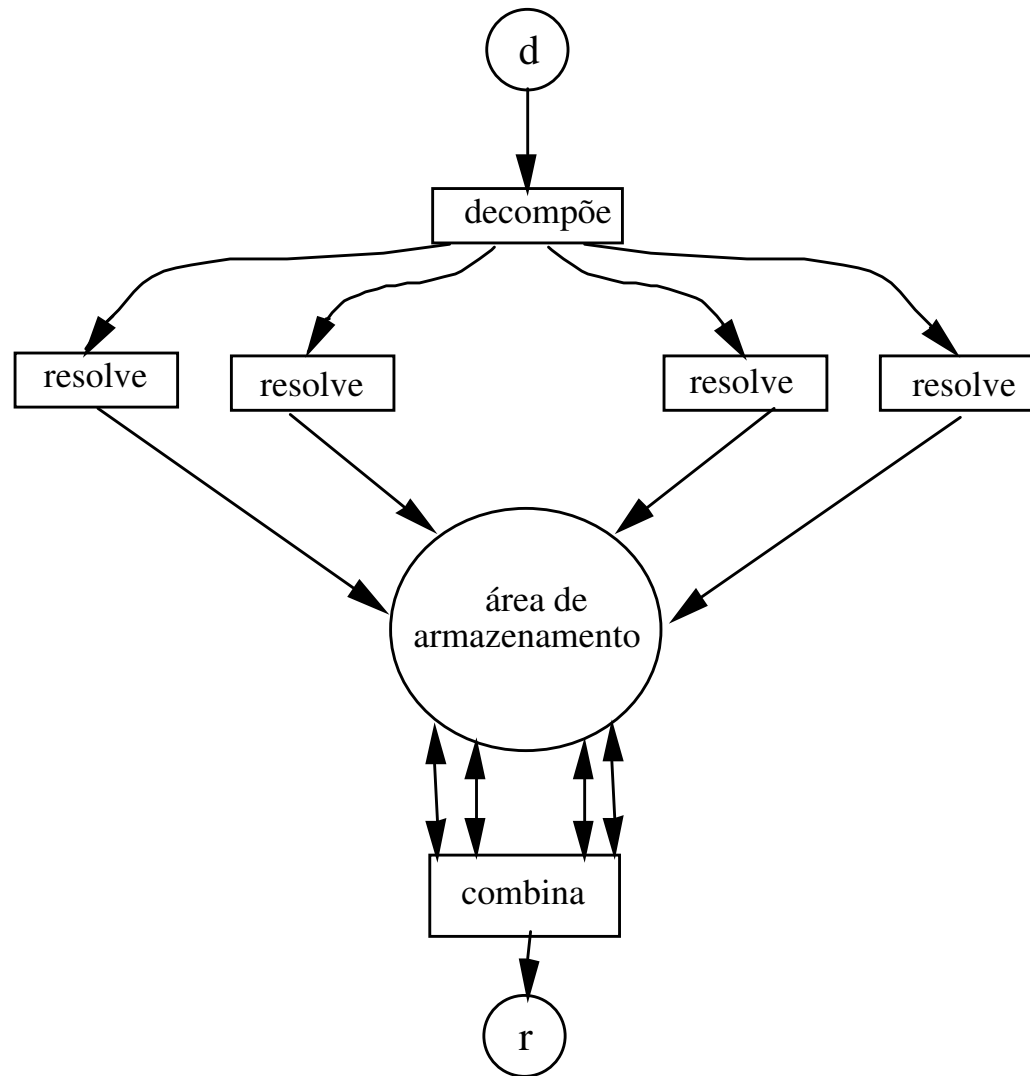
Idéias básicas da programação dinâmica

A **idéia básica** da programação dinâmica é **construir por etapas** uma resposta ótima combinando respostas já obtidas para **partes menores**.

- Inicialmente, entrada é decomposta partes mínimas, para as quais são obtidas respostas.
- Em cada passo, sub-resultados são combinados dando respostas para partes maiores, até que se obtenha uma resposta para o problema original.
- A decomposição é feita uma única vez e, além disso, os casos menores são tratados antes dos maiores.

Assim, esse método é chamado ascendente, ao contrário dos métodos recursivos, que são chamados descendentes.

O diagrama mostra a estrutura geral dessa abordagem.



Estrutura geral da programação dinâmica

A programação dinâmica **reduz** o número de possíveis combinações eliminando as que sabidamente não poderão fazer parte de uma solução ótima do problema.

É aplicável aos problemas de otimização com a seguinte **propriedade de otimalidade**.

"Uma solução não ótima de um subproblema não pode ser subsolução de solução ótima do problema".

Isso significa que só **soluções ótimas** dos subproblemas podem compor uma solução ótima do problema.

Na procura de uma solução ótima, as soluções não ótimas de subproblemas podem ser descartadas, e é esse o processo que torna os algoritmos desenvolvidos por programação dinâmica mais eficientes que os algoritmos diretos.

Além disso, os subproblemas são resolvidos uma única vez, e suas **soluções** são **guardadas** para serem usadas tantas vezes quantas for necessário.

A idéia básica da programação dinâmica sugere uma estrutura geral para algoritmos projetados por essa estratégia:

Inicialização ; Iteração ; Finalização.

Na **inicialização**, a entrada é decomposta em partes mínimas para as quais são obtidas respostas diretas.

A **iteração** vai aumentando tamanho das partes e obtendo respostas correspondentes a partir das já geradas, até que as partes atingem o tamanho da entrada original

Quando então sua **solução** é recuperada, e o processo finalizado.

Assim, chega-se ao seguinte esquema geral de algoritmo de programação dinâmica.

Algoritmo: Programação Dinâmica

Função Alg_PD($d:D$) $\rightarrow R$ {Algoritmo de programação dinâmica (abstrato)}

{ *Entrada-saída*: saída $r:R$ é resposta ótima para entrada $d:D$ }

 Incz_PD ; Iter_PD ; Fnl_PD {estrutura geral}
 onde {componentes}

 Incz_PD := {inicialização}
 entrada é decomposta em partes mínimas;
 partes mínimas dão diretamente partes da saída

 Iter_PD: itera corpo Crp_PD {iteração}
 escolhe elemento e da parte da entrada;
 combina parte da entrada com elemento e ;
 atualiza parte da saída com elemento e

 Fnl_PD: {finalização}
 extraí a saída final para a entrada original

No algoritmo Multi_Mat (para custo mínimo de produto de matrizes) apresentado no exemplo, podemos estabelecer as seguintes identificações:

Multi_Mat	Alg_PD
<u>sorte</u> D := vetor b:D de naturais	<u>sorte</u> D {entrada}
1. <u>para</u> i <u>de</u> 1 <u>até</u> n <u>faça</u> m[i, i] ← 0;	IncZ_PD {inicialização}
2. <u>para</u> u <u>de</u> 1 <u>até</u> n-1 <u>faça</u>	Iter_PD {iteração}
3. <u>para</u> i <u>de</u> 1 <u>até</u> n-u <u>faça</u>	Crp_PD {corpo da iteração}
4. j ← i+u;	
5. m[i, j] ← melhor valor;	{atualiza parte de saída}
6. <u>fim-para</u>	
7. <u>fim-para</u>	{fim da iteração}
8. <u>retorne-saída</u> (m[1, n]);	FnI_G {finalização}

Tabela : Identificações entre Multi_Mat e Alg_PD

Projeto de algoritmos por programação dinâmica

Vamos examinar um problema de **caminhos de custo mínimo em grafo orientado**. O problema consiste em determinar um caminho de custo mínimo para cada par de vértices do grafo.

Para aplicar a estratégia de programação dinâmica podemos usar um conjunto I de **vértices intermediários**, que é incrementado em cada passo.

A idéia é determinar, passo a passo, os caminhos de custo mínimo com a restrição de usar como vértices intermediários apenas os do conjunto I .

Claramente, quando o conjunto I tiver todos os vértices do grafo, não há mais restrição e teremos resolvido o problema.

Para essa idéia funcionar, devemos saber inicializar e atualizar.

Inicialmente, vejamos um exemplo concreto para ilustrar as idéias envolvidas.

Caminhos de custo mínimo em grafo orientado.

Consideremos um grafo orientado $G = \langle V, E \rangle$ com 3 vértices $V = \{v_1, v_2, v_3\}$ e a seguinte matriz de custos:

cst	v_1	v_2	v_3
v_1	0	3	6
v_2	∞	0	2
v_3	7	∞	0

Inicialmente o conjunto I de vértices intermediários é vazio e $\text{dst}_0 := \text{custo}$.

No passo 1, selecciona-se um vértice como novo vértice intermediário, digamos o vértice v_1 . Com isso obtemos um caminho de v_3 a v_2 passando por v_1 com comprimento $7 + 4 = 11 < \infty$.

dst_1	v_1	v_2	v_3
v_1	0	3	6
v_2	∞	0	2
v_3	7	11	0

No passo 2, escolhendo o vértice v_2 como novo vértice intermediário, obtemos um novo caminho de v_1 a v_3 passando por v_2 com comprimento $3+2=5 < 6$. E:

dst_2	v_1	v_2	v_3
v_1	0	3	5
v_2	∞	0	2
v_3	7	11	0

Finalmente, tomando o vértice v_3 como terceiro vértice intermediário, obtemos um caminho de v_2 a v_1 passando por v_3 com comprimento $2+7=9 < \infty$.

Assim, a **matriz de distâncias** passa a ser:

dst_3	v_1	v_2	v_3
v_1	0	3	5
v_2	9	0	2
v_3	7	11	0

Essa matriz fornece o custo mínimo de caminhos entre cada par de vértices.

Distância entre vértices em grafo orientado.

Consideremos um grafo orientado G , com conjuntos

- conjunto $V = \{v_1, \dots, v_n\}$ de vértices e
- matriz de custo (dando arestas valoradas).

Desejamos determinar a distância (custo do caminho mínimo) entre cada par de vértices.

Usaremos um conjunto $I \subseteq V$ (de possíveis vértices intermediários), que será incrementado em cada passo.

Inicializamos o conjunto I de vértices intermediários como vazio ($I := \emptyset$) e a matriz de distâncias com a matriz de custos: $\text{dist}_I := \text{custo}$.

Em cada passo, seleciona-se um novo vértice $w \in (V - I)$ como novo vértice intermediário ($I := I \cup \{w\}$) e atualiza-se a matriz de distâncias (comparando cada valor atual com o de um caminho passando pelo novo vértice intermediário w).

Esse processo prossegue enquanto houver possíveis novos vértices intermediários ($I \neq V$).

Ao final $I = V$ e teremos o custo do caminho mínimo entre cada par de vértices.

O **algoritmo Dist_vrt** foi obtido pela de programação dinâmica observando:

- As linhas de 1 a 7 fazem a inicialização: a linha 1 inicializa as partes mínimas da entrada (sem vértices intermediários) e a linhas de 2 a 7 inicializam a saída correspondente.
- A iteração (aumentando o conjunto de possíveis vértices intermediários) está nas linhas de 8 a 16: a linha 9 seleciona um novo elemento, que é usado para atualizar a parte I da entrada (na linha 10) e a saída correspondente (nas linhas de 11 a 15).
- A finalização está na linha 17, quando se extrai a solução.

Algoritmo: Custo mínimo de caminhos entre vértices de grafo orientado

Função Dist_vrt($d:D$) $\rightarrow R$ {distância entre vértices de grafo}

{ *Entrada:* grafo orientado G com conjunto V de vértices e matriz $cst:M$ (custos)

Saída: matriz $dst:R$ (custos dos melhores caminhos entre vértices de G) }

sorte $D :=$ Grafos valorados com custos ; $R := M$; $Vrt :=$ Vértices ;

$P :=$ Conj[Vrt] {conjuntos de vértices} ; $M :=$ matriz [$u \in V, v \in V$] de $\mathbb{R}^*$;

var {auxiliares} $I:P$ {parte da entrada} ; $w:Vrt$ {vértice} ;

1. $I \leftarrow \emptyset$;	{inicializa conjunto I sem intermediários}
2. <u>para cada</u> $u \in V$ <u>faça</u>	{inicializa distância sob I: 7}
3. $\text{dst}[u, u] \leftarrow 0$;	{inicializa diagonal com zero}
4. <u>para cada</u> $v \in (V - \{u\})$ <u>faça</u>	{inicializa não diagonal: 7}
5. $\text{dst}[u, v] \leftarrow \text{cst}[u, v]$;	{distância inicial como custo}
6. <u>fim-para</u>	{4: cada $v \in (V - \{u\})$ }
7. <u>fim-para</u>	{2: cada $u \in V$ }
8. <u>enqto</u> $I \neq V$ <u>faça</u>	{itera: 16}
9. escolha $w \in (V - I)$;	{seleciona novo vértice}
10. $I \leftarrow I \cup \{w\}$;	{atualiza I com novo intermediário}
11. <u>para cada</u> $u \in V$ <u>faça</u>	{atualiza distância sob I: 15}
12. <u>para cada</u> $v \in (V - \{u\})$ <u>faça</u>	{atualiza não diagonal: 14}
13. $\text{dst}[u, v] \leftarrow \min(\text{dst}[u, v], \text{dst}[u, w] + \text{dst}[w, v])$;	
14. <u>fim-para</u>	{12: cada $v \in (V - \{u\})$ }
15. <u>fim-para</u>	{11: cada $u \in V$ }
16. <u>fim-enqto</u>	{8: $I \neq V$ (todos os vértices)}
17. <u>retorne-saída</u> (dst) ;	{dá como saída resposta pronta}
18. <u>fim-Função</u>	{fim do algoritmo Dist_vrt: distância entre vértices}

Pode-se determinar um caminho de custo mínimo, guardando-se os vértices correspondentes ao mínimo na linha 13 do algoritmo.

O problema do projeto de sistemas confiáveis (PSC).

O objetivo é construir um sistema com **máxima confiabilidade**, restrito aos recursos orçamentários disponíveis e pode ser enunciado como segue.

O sistema desejado consiste da ligação em série de **n** dispositivos:

$$D_1, D_2, \dots, D_n.$$

Cada dispositivo D_i tem uma confiabilidade b_i , para $i = 1, \dots, n$.

Para aumentar a confiabilidade do sistema, serão usadas **cópias** reservas de cada dispositivo, as quais entram em ação imediatamente quando o dispositivo em uso deixar de funcionar.

Cada cópia de um dispositivo D_i tem um custo $\text{custo}(i)$, e o montante **M** de reserva orçamentária para construir o sistema é um limitante do problema.

O problema consiste em **maximizar a confiabilidade** do sistema sem ultrapassar o limite de custo pré-determinado M .

O sistema está constituído de n estágios, cada estágio se referente a um dispositivo. Consideremos o dispositivo D_i (no estágio i).

Sejam:

- n_i o número de cópias disponíveis do dispositivo;
- b_i a confiabilidade (probabilidade de funcionamento) do dispositivo.

Assim, temos

- $(1 - b_i)$ é a probabilidade de D_i não funcionar, e
- $(1 - b_i)^{n_i}$ é a probabilidade de n_i cópias do dispositivo i não funcionarem.

Portanto, a **confiabilidade $cfb(i)$ do estágio i** pode ser dada por $cfb(i) = 1 - (1 - b_i)^{n_i}$.

O sistema terá confiabilidade $\prod_{i=1}^n cfb(i)$ e custo $\sum_{i=1}^n [custo(i) \cdot n_i]$.

O problema é maximizar a confiabilidade do sistema, sujeito ao limite de custo M , isto é

$$\text{Máx } \prod_{i=1}^n cfb(i) \quad \text{sujeito a } \sum_{i=1}^n [custo(i) \cdot n_i] \leq M.$$

Trata-se de um **problema de otimização com restrições**. A programação dinâmica é um método apropriado para resolver de forma exata e com bom desempenho tais problemas.

Um algoritmo trivial analisaria todos os casos possíveis. A programação dinâmica vai eliminar aqueles casos que sabidamente não estarão na solução ótima.

O montante M é a principal limitação. É preciso comprar pelo menos um exemplar de cada dispositivo, e o restante pode ser gasto em exemplares extras de dispositivos.

Assim, pode-se calcular o número máximo m_i de exemplares de cada dispositivo da seguinte forma:

$$m_i = \left\lfloor \frac{\left(M - \sum_{j=1}^n \text{custo}(j) \right) + \text{custo}(i)}{\text{custo}(i)} \right\rfloor$$

O problema PSC com n estágios é decomposto em problemas de 1 estágio, isto é, em que só existe o dispositivo D_1 , que pode ter no máximo m_1 exemplares, o que significa m_1 problemas: cada um, com um custo e uma confiabilidade.

A cada passo, o tamanho do problema **crescerá de 1** (mais um dispositivo é considerado).

A solução de problemas, num certo passo, usa a solução de passos anteriores. Assim, em $n-1$ passos, é obtido e solucionado o problema original.

Para cada $i = 1, \dots, n$, consideremos:

S_i , consistindo do conjunto dos possíveis pares confiabilidade e custo do projeto com i estágios, usando j exemplares do dispositivo i ($j = 1, 2, \dots, m_i$), com custo que não ultrapasse o montante M .

Uma configuração com maior confiabilidade e menor custo é preferível desde que viável.

Assim, temos a seguinte regra de dominância para preterir configurações.

Sendo (x', y') e (x'', y'') pares viáveis ($y' \leq M$ e $y'' \leq M$): (x', y') domina (x'', y'') sempre que $x' > x''$ e $y' \leq y''$.

Aplicando essa **regra de dominância**, podemos otimizar S_i , eliminando pares dominados, para obter C_i . Ou seja:

$$C_i := \{(x, y) \in S_i / \nexists (x', y') \in S_v \text{ tal que } (x', y') \text{ domina } (x, y)\}.$$

Assim, esse conjunto otimizado C_i é obtido em dois passos:

- no primeiro passo, constrói-se S_i , com todos os pares (x, y) de confiabilidade e correspondente custo, considerados i estágios e o limitante M ;
- no segundo passo, S_i é otimizado, por eliminação dos pares dominados.

Os conjuntos S_i de pares, para $i = 1, \dots, n$, podem ser calculados indutivamente da seguinte forma.

Para $i = 1$, temos S_1 diretamente, pois

$$S_1 = \{(1 - (1 - b_1))^j, j \cdot \text{custo}(1) / j = 1, 2, \dots, m_1\}.$$

Para $i > 1$, podemos obter S_i a partir de C_{i-1} , pois

$$S_i := \{(x \cdot \text{cfb}(i))^j, y + j \cdot \text{custo}(i) / (x, y) \in C_{i-1} \text{ tal que } y + j \cdot \text{custo}(i) \leq M, j = 1, \dots, m_i\}.$$

Consideremos o **PSC** com montante $M=190$ e $n=4$ dispositivos, tendo confiabilidades e custos dados pela tabela abaixo

i	1	2	3	4
b_i	0.9	0.8	0.7	0.9
custo(i)	30	35	40	22

Nesse caso, o número máximo de exemplares de cada dispositivo resulta $m_1=3$; $m_2=2$; $m_3=2$ e $m_4=3$.

Obtemos as seguintes seqüências de conjuntos de pares confiabilidade e custo.

Para $i=1$, temos:

$S_1 = \{(0.9, 30), (0.99, 60), (0.999, 90)\}$., como não há dominâncias, obtemos

$C_1 = S_1 = \{(0.9, 30), (0.99, 60), (0.999, 90)\}$.

Para $i=2$,

$S = \{(0.72, 65), (0.792, 95), (0.7992, 125), (0.864, 100), (0.9504, 130), (0.9599, 160)\}$

Notamos que há dominância de pares: de $(0.7992, 125)$ por $(0.864, 100)$, assim:

$C_2 = \{(0.72, 65), (0.792, 95), (0.864, 100), (0.9504, 130), (0.9599, 160)\}$.

Para $i=3$,

$$S_3 = \{(0.504, 105), (0.6552, 145), (0.5544, 135), (0.66528, 175), (0.6048, 140), (0.78624, 180), (0.66528, 170)\}.$$

Como não há dominâncias, temos:

$$C_3 = S_3 = \{(0.504, 105), (0.6552, 145), (0.5544, 135), (0.66528, 175), (0.6048, 140), (0.78624, 180), (0.66528, 170)\}.$$

Para $i=4$,

$$S_4 = \{(0.4536, 127), (0.49896, 149), (0.5039496, 171), (0.49896, 157), (0.548856, 179), (0.54432, 162), (0.598752, 184), (0.58968, 167), (0.698648, 189)\}.$$

Notamos as seguintes dominância de pares:

- de $(0.5039496, 171)$ e $(0.548856, 179)$ por $(0.58968, 167)$;
- de $(0.49896, 157)$ por $(0.49896, 149)$.

Eliminando esses pares dominados, obtemos:

$$C_4 = \{(0.4536, 127), (0.49896, 149), (0.54432, 162), (0.598752, 184), (0.58968, 167), (0.698648, 189)\}.$$

Portanto, o **sistema ótimo** é o que tem **confiabilidade 0.698648** e **custo 189**.

Agora, podemos obter um algoritmo de programação dinâmica para o problema PSC.

Algoritmo: Projeto de sistemas com confiabilidade máxima

Função PSC($b: \text{Cnf}$, $\text{custo}: \text{Cst}$, $M: \mathbb{R}$) $\rightarrow [0, 1]$ {confiabilidade máxima}
 { *Entrada*: vetores b (com a confiabilidade) e custo (com o custo) de cada dispositivo D_i (para $i = 1, \dots, n$) e montante (cota superior para custo)
Saída: r (confiabilidade máxima de sistema com n dispositivos) }

sorteCnf := vetor [1..n] de [0, 1]; Cst := vetor [1..n] de $\mathbb{R}$
 Par := [0, 1] $\times$ $\mathbb{R}$ {auxiliar: confiabilidade e custo};
var {auxiliares} cst_min: $\mathbb{R}$ {custo mínimo}; m : vetor [1..n] de $\mathbb{N}$
 {número máximo de cópias}; S, C: Conj[Par] {conjuntos S_i e C_i }; x' : [0, 1] {confiabilidade}; y' : $\mathbb{R}$ {custo};

1. $\text{cst_min} \leftarrow \sum_{j=1}^n \text{custo}[j]$; {custo mínimo}
2. para i de 1 até n faça $m[i] \leftarrow \left\lfloor \frac{M - \text{cst_min}}{\text{custo}[i]} \right\rfloor + 1$; {nº máx. de cópias do dispos. i }

3. $S \leftarrow \{ ((1 - (1 - b[1]))^j), j \cdot \text{custo}[1] \} / j = 1, 2, \dots, m[1] \};$	{conjunto S_1 }
4. $C \leftarrow S;$	{conjunto C_1 }
5. $i \leftarrow 1;$	{inicializa controle i }
6. <u>enqto</u> $i < n$ <u>faça</u>	{estágios: de 6 a 17}
7. $i \leftarrow i + 1;$	{incrementa controle i }
8. $S \leftarrow \emptyset;$	{inicializa S_{i-1} }
9. <u>para cada</u> $(x, y) \in C$ <u>faça</u>	{varre C_{i-1} : de 9 a 15}
10. <u>para</u> j <u>de</u> 1 <u>até</u> $m[i]$ <u>faça</u>	{número de cópias: de 10 a 14}
11. $x' \leftarrow x \cdot (1 - (1 - b[1]))^j;$	{confiabilidade}
12. $y' \leftarrow y + j \cdot \text{custo}[i];$	{custo}
13. <u>se</u> $y' \leq M$ <u>então</u> $\text{Insere}((x', y'), S);$	{atualiza S_j }
14. <u>fim-para</u>	{10: j de 1 até $m[i]$ }
15. <u>fim-para</u>	{9: cada $(x, y) \in C$ }
16. $C \leftarrow S$ - pares dominados;	{obtem C_i de S_i }
17. <u>fim-enqto</u>	{6: $i \leq n$ }
18. $r \leftarrow \text{Máx}\{x / \exists y: (x, y) \in C\};$	{confiabilidade máxima em C^n }
19. <u>retorne-saída</u> (r);	{dá saída}
20. <u>fim-Função</u>	{fim do algoritmo PSC: confiabilidade máxima de sistema}

Complexidade de algoritmos de programação dinâmica

Vamos ver como se pode efetuar a **análise de complexidade pessimista** de alguns algoritmos típicos desenvolvidos por programação dinâmica.

Complexidade de custo mínimo de produto de matrizes.

Considere o algoritmo **Multi_Mat** (custo mínimo de produto de matrizes).

- A iteração de inicialização na linha 1 é executada n vezes.
- A iteração nas linhas de 2 a 7 é executada $n-1$ vezes.
- Já o número de vezes que é executada a iteração nas linhas de 3 a 6 (para i de 1 até $n-u$) dependendo de u (o índice da iteração externa de 2 a 7).
- O mínimo na linha 5 é tomado sobre $j-i$ (i. e. u) elementos.

Assim, a complexidade da iteração das linhas de 2 a 7 é dada por $\sum_{u=1}^{n-1} [u \cdot (n-u)]$.

Mas, $\sum_{u=1}^{n-1} [u \cdot (n-u)]$ tem ordem $\Theta(n^3)$.

Portanto, a complexidade do algoritmo Multi_Mat tem ordem

$$c_p[\text{Multi_Mat}](n) = O(n + n^3) = \mathbf{O(n^3)}.$$

Agora, vamos fazer uma análise mais cuidadosa e detalhada, usando as idéias vistas na seção 3.2 do capítulo 3 (cf. 3.2.2, 3.2.4 e 3.2.5).

Esse algoritmo Multi_Mat recebe como **entrada**

um vetor b : vetor $[0..n]$ de $\mathbb{N}$ (com as dimensões das n matrizes a multiplicar)

e fornece como **saída**

um natural $m[1, n]$ (o número mínimo de multiplicações para o produto),

usando como área auxiliar de armazenamento

uma matriz m : M , sendo $M :=$ matriz $[1..n, 1..n]$ de $\mathbb{N}$.

Aqui, é natural tomar como tamanho da entrada o número n de matrizes a multiplicar.

Nesse caso, parece natural tomar como fundamental as operações aritméticas sobre naturais, incluindo inicializações com 0.

Como o algoritmo Multi_Mat é uma seqüência, podemos aplicar os resultados vistos na seção 3.2 (cf. 3.2.2).

O desempenho de **Multi_Mat** sobre uma entrada b tem contribuições dadas por suas componentes: **Inicialização, Iteração e Finalização**.

Para determinar essas contribuições, devemos obter os valores das entradas de cada trecho.

A inicialização (linha 1) fornece valores iniciais à diagonal principal da variável $m:M$ {custos das partes mínimas da entrada}

$$[m_0[1,1], \dots, m_0[n,n]] := [0, \dots, 0] \quad \{\text{tamanho } \dim(m_0) = n\}$$

A inicialização (linha 1) executa n vezes o comando $m[i,i] \leftarrow 0$.

Esse comando $m[i,i] \leftarrow 0$ tem desempenho $desemp[m[i,i] \leftarrow 0] = 1$.

Assim, **inicialização Incz_Multi_Mat** tem

$$desemp[\text{Multi_Mat}](b) = n \cdot 1 = n;$$

o que resulta em complexidade pessimista

$$c_p^{\leq}[\text{Incz_Multi_Mat}](n) = n.$$

A seguir, a iteração nas linhas de 2 a 7 (para u de 1 até $n-1$ faça) repete $n-1$ vezes o corpo (tratando das diagonais superiores e aumentando o número de matrizes que serão multiplicadas).

Desse modo, são gerados novos valores para a variável $m:M$ {custos das partes das diagonais superiores}.

Os valores sucessivos da variável $m:M$ têm tamanhos como se segue:

$$\begin{array}{c|cccccc} u & 0 & 1 & \dots & u & \dots & n-1 \\ \dim(m_u) & n & n-1 & \dots & n-u & \dots & 1 \end{array}$$

Assim, ao início da $u^{\text{ésima}}$ iteração, a variável $m:M$ tem tamanho $\dim(m_u) = n - u$.

Agora, o corpo **Crp_Multi_Mat** é a iteração definida nas linhas de 3 a 6 varrendo cada posição na diagonal. Aqui, só contribui para o desempenho a linha 5

$$m[i, j] \leftarrow \min_{i \leq k < j} \{ (m[i, k] + m[k+1, j]) + (b[i-1] \times b[k] \times b[j]) \}.$$

Essa linha 5 envolve uma **minimização** sobre k variando de i até $j-1 = i+u-1$ (cf. linha 4): uma minimização sobre um conjunto C_{ij} com $(j-1) - (i-1) = u$ elementos.

O k^{o} elemento desse conjunto C_{ij} é o natural dado pela expressão $(m[i, k] + m[k+1, j]) + (b[i-1] \times b[k] \times b[j])$; assim sua determinação envolve 2 somas e 2 produtos de naturais.

Portanto, a determinação de todos os u elementos desse conjunto C_u envolve $4 \cdot u$ operações aritméticas.

Assim, o desempenho da minimização da linha 5 na $u^{\text{ésima}}$ iteração é dado por

$$\text{desemp}[\text{linha 5}](m_u) = 4 \cdot u.$$

Desse modo, na $u^{\text{ésima}}$ iteração, as linhas 4 e 5 têm os seguintes desempenhos:

Comando	desempenho
4. $j \leftarrow i + u;$	1
5. $m[i, j] \leftarrow \text{melhor valor};$	$4 \cdot u$

Assim, temos $\text{desemp}[\text{linhas 4-5}](m_u) = 1 + 4 \cdot u$.

A iteração definida nas linhas de 3 a 6 (para de 1 até $n-u$ faça) repete $n-u$ as linhas 4 e 5.

Portanto, o desempenho do corpo **Crp_Multi_Mat** na $u^{\text{ésima}}$ iteração é dado pela somatória das $n-u$ contribuições:

$$\text{desemp}[\text{Crp_Multi_Mat}](m_u) = (1 + 4 \cdot u) \cdot (n - u).$$

A iteração `Iter_Multi_Mat` (linhas de 2 a 7) repete $n-1$ vezes o corpo `Crp_Multi_Mat`.

Portanto, o desempenho da iteração `Iter_Multi_Mat` é dado pela somatória das $n-1$ contribuições:

$$\begin{matrix} u=1 & & U & & u=n-1 \\ (1+4.1).(n-1) & + \dots + & (1+4.u).(n-u) & + \dots + & (1+4.(n-1)).1 \end{matrix}$$

Assim, o desempenho da iteração `Iter_Multi_Mat` é dado por

$$desemp[Iter_Multi_Mat](m_0) = \sum_{u=1}^{n-1} [(1+4.u).(n-u)].$$

Desse modo, pode-se ver que a complexidade pessimista da iteração `Iter_Multi_Mat` tem ordem cúbica.

$$c_P^{\leq}[Iter_Multi_Mat](n) = \Theta(n^3).$$

Agora, a ordem da **complexidade pessimista** do algoritmo Multi_Mat é a soma das três contribuições devidas à inicialização, à iteração e à finalização.

$$\begin{array}{rcl}
 c_p^{\leq} [\text{Incz_Multi_Mat}](n) & = & n \\
 + & & \\
 c_p^{\leq} [\text{Iter_Multi_Mat}](n) & = & \Theta(n^3) \\
 + & & \\
 c_p^{\leq} [\text{Fnl_Multi_Mat}](n) & = & 0 \\
 \hline
 c_p^{\leq} [\text{Multi_Mat}](n) & = & \Theta(n+n^3)
 \end{array}$$

Assim, a complexidade pessimista de Multi_Mat tem a ordem

$$c_p^{\leq} [\text{Multi_Mat}](n) = \Theta(n^3).$$

Logo, o algoritmo **Multi_Mat tem complexidade pessimista** cúbica:

$$c_p^{\leq} [\text{Multi_Mat}](n) = \Theta(n^3).$$

Complexidade de distância entre vértices em grafo orientado.

Considere o algoritmo Dist_vrt para distância (custo mínimo de caminhos) entre pares de vértices em grafo orientado.

- A inicialização do conjunto I ($I \leftarrow \emptyset$ na linha 1) tem complexidade constante.
- As linhas de 2 a 7 inicializam cada um dos n^2 elementos da matriz dst.

Assim, a complexidade da inicialização (linhas de 1 a 7) tem ordem $O(n^2)$.

- A iteração das linhas de 8 a 16 é executada até $I=V$: n vezes.
- A parte dominante de seu corpo (linhas de 9 a 15) é devida às linhas de 11 a 15, atualizando cada um dos n^2 elementos da matriz dst, com complexidade de ordem $O(n^2)$.

Logo, a complexidade da iteração tem ordem $O(n \cdot n^2) = O(n^3)$.

Portanto, a **complexidade** do algoritmo Dist_vrt tem ordem

$$c_p^{\leq}[\text{Dist_vrt}](n) = O(n^2 + n^3) = \mathbf{O(n^3)}.$$

Agora, vamos fazer uma análise mais cuidadosa e detalhada, usando as idéias vistas na seção 3.2 do capítulo 3 (cf. 3.2.2, 3.2.4 e 3.2.5).

Esse **algoritmo Dist_vrt** recebe como **entrada** $d:D$

um grafo orientado G com conjunto V de vértices e matriz cst de custos e fornece como **saída**

uma matriz dst (com os custos dos melhores caminhos entre vértices de G)

Consideremos um grafo orientado G , com conjunto $V = \{v_1, \dots, v_n\}$ de vértices.

É natural, para tamanho da entrada o número n de vértices de G .

E parece natural tomar como **operações fundamentais** as manipulações com conjuntos (de vértices) e matrizes.

Como o algoritmo $Dist_vrt$ é uma seqüência, podemos aplicar os resultados vistos na seção 3.2 (cf. 3.2.2).

O desempenho de $Dist_vrt$ sobre uma entrada $d:D$ tem contribuições dadas por suas componentes:

Inicialização, Iteração e Finalização.

Para determinar essas contribuições, devemos obter os valores das entradas de cada trecho.

A inicialização (linhas de 1 a 7) fornece valores iniciais às variáveis $l:P$ {parte da entrada} e $dst:R$ {custos dos melhores caminhos entre vértices de G }

$$l_0 := \emptyset \quad \{\text{tamanho} \mid |l_0| = 0\}$$

$$dst_0[u,v] := \begin{cases} 0 & (\text{se } u=v) \\ cst[u,v] & (\text{se } u \neq v) \end{cases} \quad \{\text{tamanho } \dim(dst_0) = n^2\}$$

Assim, a saída da inicialização é $Inc_{z_Dist_vrt}(G) = (l_0, dst_0)$.

O **desempenho da inicialização** tem as seguintes contribuições:

comando	nº de vezes	esforço
1. $I \leftarrow \emptyset;$	1	1
2. <u>para cada</u> $u \in V$ <u>faça</u>	n	
3. $\text{dst}[u, u] \leftarrow 0;$	1	1
4. <u>para cada</u> $v \in (V - \{u\})$ <u>faça</u>	$(n - 1)$	
5. $\text{dst}[u, v] \leftarrow \text{cst}[u, v];$	1	1
6. <u>fim-para</u>		
7. <u>fim-para</u>		

Assim,

$$\text{desemp}[\text{Incz_Dist_vrt}](G) = (1 \cdot 1) + (n \cdot 1) + (n \cdot (n - 1) \cdot 1) = n^2 + 1.$$

Agora, examinemos a iteração enqto $I \neq V$ (linhas de 8 a 16).

A cada iteração do corpo (linhas de 9 a 15), um novo vértice $w \in (V - I)$ é selecionado e incluído em I (linhas 9 e 10), sendo atualizados os valores da matriz $\text{dst} : R$ (linhas de 11 a 15).

Desse modo, são gerados novos valores para as variáveis $I:P$ e $dst:R$ através de valores da variável $w:Vrt \{seleção\}$

$$\begin{array}{l|l}
 w:Vrt & w_1 \in (V - I_0) \quad \dots \quad w_k \in (V - I_{k-1}) \quad \dots \quad w_n \in (V - I_{n-1}) \\
 I:P & I_1 := I_0 \cup \{w_1\} \quad \quad \quad I_k := I_{k-1} \cup \{w_k\} \quad \quad \quad I_n := I_{n-1} \cup \{w_n\} \\
 dst:R & dst_1 \quad \quad \quad \dots \quad dst_k \quad \quad \quad \dots \quad dst_n
 \end{array}$$

Assim, os valores sucessivos das variáveis $I:P$ e $dst:R$ têm tamanhos como se segue:

$$\begin{array}{l|l}
 j & 0 \quad 1 \quad \dots \quad k \quad \dots \quad n \\
 |I_j| & 0 \quad 1 \quad \dots \quad k \quad \dots \quad n \\
 \dim(dst_j) & n^2 \quad n^2 \quad \dots \quad n^2 \quad \dots \quad n^2
 \end{array}$$

Logo, a iteração é executada para $|I|$ variando de 0 a $n-1$: n vezes.

Ao início da $k^{ésima}$ iteração, as variáveis $I:P$ e $dst:R$ têm tamanhos $|I_k| = k$ e $\dim(dst_k) = n^2$.

O desempenho do corpo **Crp_Dist_vrt** (linhas de 9 a 15) em cada iteração é dado pela soma das contribuições das linhas 9 e 10 e das linhas de 11 a 15.

Na k ésima iteração, as linhas 9 e 10 atuam sobre as variáveis

- $I:P$ e $dst:R$ com tamanhos $|I_k|=k$ e
- $\dim(dist_i)=n^2$ e gerando novo valor para $I:P$ com tamanho $|I_{k+1}|=k+1$.

Assim, as linhas 9 e 10 dão as seguintes contribuições para o desempenho:

comando	desempenho
9. escolha $w \in (V-I)$;	1
10. $I \leftarrow I \cup \{w\}$;	k

Assim, temos $desemp[linhas\ 9,10](I_k, dst_k) = 1 + k = k + 1$.

Na $k^{\text{ésima}}$ iteração, as linhas de 11 a 15 atuam sobre variáveis:

- $p:P$ e $\text{dist}:V$ com tamanhos $|I_{k+1}| = k+1$ e
- $\dim(\text{dst}_k) = n^2$.

Assim, as linhas de 11 a 15 dão as seguintes contribuições para o desempenho:

comando	nº de vezes	esforço
11. <u>para cada</u> $u \in V$ <u>faça</u>	n	
12. <u>para cada</u> $v \in (V - \{u\})$ <u>faça</u>	$(n-1)$	
13. $\text{dst}[u,v] \leftarrow \min(\text{dst}[u,v], \text{dst}[u,w] + \text{dst}[w,v])$		1
14. <u>fim-para</u>		
15. <u>fim-para</u>		

Assim, $\text{desemp}[\text{linhas 11-15}](I_{k+1}, \text{dst}_k) = n \cdot (n-1) \cdot 1 = n^2 - n$.

Desse modo, o desempenho do corpo Crp_Dist_vrt na $k^{\text{ésima}}$ iteração é dado por:

$$\begin{aligned} \text{desemp}[\text{Crp_Dist_vrt}](l_k, \text{dst}_k) &= \\ &= \text{desemp}[\text{linhas 9,10}](l_k, \text{dst}_k) + \text{desemp}[\text{linhas 11-15}](l_{k+1}, \text{dst}_k) \\ &= (k+1) + n \cdot (n-1) = n \cdot (n-1) + (k+1). \end{aligned}$$

Agora, como a iteração enqto $l \neq V$ (linhas de 8 a 16) é executada n vezes, para $|l|$ variando de 0 a $n-1$, seu desempenho é a somatória das n contribuições:

$$\begin{array}{ccccccc} j=0 & & & j=k & & & j=n-1 \\ n \cdot (n-1) + 1 & + & \dots & + & n \cdot (n-1) + (k+1) & + & \dots & + & n \cdot (n-1) + n \end{array}$$

Assim, para o desempenho da iteração **Iter_Dist_vrt** temos:

$$\begin{aligned} \text{desemp}[\text{Iter_Dist_vrt}](l_0, \text{dst}_0) &= \\ &= [n \cdot (n-1) + 1] + \dots + [n \cdot (n-1) + (k+1)] + \dots + [n \cdot (n-1) + n] = \\ &= n \cdot [n \cdot (n-1)] + \sum_{k=0}^{n-1} (k+1) = [n^2 \cdot (n-1)] + [n \cdot (n-1)] / 2. \end{aligned}$$

Agora, o desempenho do algoritmo `Dist_vrt` é a soma das duas contribuições:

$desemp[Inc_{Dist_vrt}](G)$ e $desemp[Iter_{Dist_vrt}](Inc_{Dist_vrt}(G))$:

$$\begin{aligned} desemp[Dist_vrt](I_k, dst_k) &= \\ &= desemp[Inc_{Dist_vrt}](G) + desemp[Iter_{Dist_vrt}](I_0, dst_0) \\ &= [n^2 + 1] + [n \cdot (n-1) \cdot (n+1/2)]. \end{aligned}$$

Assim, para a complexidade pessimista de `Dist_vrt` temos

$$c_p^{\leq}[Dist_vrt](n) = O([n^2 + 1] + [n^3 - n^2/2 - n/2]) = \Theta(n^3).$$

Logo, o **algoritmo `Dist_vrt` tem complexidade pessimista** cúbica:

$$c_p^{\leq}[Dist_vrt](n) = \Theta(n^3).$$